

Topologisches Traversierungsframework

Rekursive Iteration v0.3: Optionale Geometrie, Phase-Ladder, Anti-Drift,
Constraint-Lattice, Dual-Fabric und post-symbolische Collapse-Maschine

Sebastian Klemm
Independent Researcher, Germany

Version 0.3 — rekursive Synthese, Mai 2026

Zusammenfassung

Dieses Dokument ersetzt und absorbiert die vorherige Fassung des Topologischen Traversierungsframeworks. Die Fassung v0.2 definierte den Hypercube als Puzzle-Raum: ein strukturiertes, zunächst blankes Möglichkeitsfeld, dessen Dimensionen, Constraints, Beobachtungen und Normen sukzessive belegt werden, bis der Lösungsraum deterministisch oder quasi-deterministisch kollabiert. Die vorliegende Iteration erweitert diesen Kern durch drei neue, chirurgisch aus den vorliegenden Quellen neutralisierte Mechanismen: erstens eine *Geometrie der Optionalität*, in der Wahlräume nicht nur als vorhandene Optionen, sondern als tatsächlich begehbare Pfade modelliert werden; zweitens eine *Phase-Ladder-Architektur*, in der operative Last zwischen phasenversetzten Trägern migriert, sobald Friktion, Schock oder Sättigung kritische Schwellen überschreiten; drittens einen *Mass-Control-Kalkül*, der nicht als Domäne der Beeinflussung übernommen wird, sondern als formale Quelle für Operatorregistries, Constraint-Lattices, Dual-Fabric-Konsistenz, Gating, Fail-Closed-Verhalten, Emissionsaudit und adversarielle Robustheit.

Das Ergebnis ist eine erweiterte Spezifikation für eine post-symbolische topologische Maschine. Diese Maschine kann beliebige strukturierbare Systemräume als Hypercube, HDAG, Constraint-Komplex oder Carrier-Ladder modellieren. Sie erkennt Pfadexzisionen, Optionalitätsverengungen, Phasenraumkrümmungen, Sättigungspunkte, Carrier-Migrationsfenster, negative Topologie, Multi-Lösungsräume und Kollapsschwellen. Sie generiert nicht einfach Artefakte, sondern erzeugt auditierte Strukturentscheidungen: jeder relevante Übergang muss projiziert, gelöst, gated, falsifiziert, replaybar gemacht und als Crystal oder CrystalSet committed werden. Damit wird das Framework von einer Generations- und Wissenserschließungsarchitektur zu einem allgemeinen Instrument für Strukturdiagnose, Modellbildung, Prozessplanung, Resilienzdesign, Compliance, Forschung, Software-Synthese und defensive Analyse komplexer Felder.

Die Spezifikation bleibt ausdrücklich domänenneutral. Begriffe wie Phase-Ladder, Carrier, Optionalität, Pfadexzision, Krümmung, Anti-Drift, Dual-Fabric und Butterfly-Leverage werden nicht in ihrer ursprünglichen politischen, ökonomischen oder feldtheoretischen Domäne übernommen. Sie werden als technische Muster abstrahiert: als Graphoperationen, Constraints, Zustandsübergänge, Gating-Regeln, Evidenzschemata und Replay-Bedingungen. Ziel ist ein professionelles, implementierungsnahes Werk, das Overleaf-tauglich ist und als Grundlage für Rust-Module, Agentenprompts, Architekturentscheidungen und weitere Forschung dienen kann.

Inhaltsverzeichnis

1	Executive Summary	2
2	Quellenneutralisierung und chirurgische Extraktion	2
2.1	Methode	2
2.2	Extraktionstabelle	3

2.3	Was ausdrücklich nicht übernommen wird	3
3	Das Grundmodell: Vom Puzzle-Cube zum Feld-Cube	3
3.1	Hypercube, Pfadraum und Carrier-Raum	3
3.2	Vier Raumebenen	4
4	Optionalitätsgeometrie	4
4.1	Formale und operative Optionen	4
4.2	Pfadexzision	5
4.3	Kompression, Asymmetrie, Verstärkung, Konvergenz	5
5	Phase-Ladder und Carrier-Migration	5
5.1	Carrier als Träger operativer Last	5
5.2	Carrier-Migration	6
5.3	Anwendungsfälle im Framework	6
5.4	Anti-Drift	7
6	Tri-temporale Architektur	7
7	Constraint-Lattice, Resonite, Infogene	8
7.1	Resonite als messbarer Constraint-Kern	8
7.2	Constraint-Lattice	8
7.3	Infogene und Infogenome	8
8	WEAVE: Deterministische Key-Space-Scherung	9
9	Dual-Fabric und Stitcher-Mirror	9
9.1	Warum eine einzige Sicht nicht reicht	9
9.2	Stitcher und Mirror	9
10	Der erweiterte Solve-Gate-Coagula-Zyklus	10
10.1	Nukleus	10
10.2	Projektionsphase	10
10.3	Solve	10
10.4	Gate	10
10.5	Coagula	11
11	51-Prozent-Kollaps als Constraint-Perkolation	11
11.1	Warum die Zahl nicht naiv gelesen werden darf	11
11.2	Collapse Certificate	12
12	Negative Topologie und Wissenserschliessung	12
12.1	Unwissen als Form	12
12.2	Betti-Lücken	12
13	Multi-Solution-Handling	12
14	Operator Registry und Status-Taxonomie	13
14.1	Operator Registry	13
14.2	Status-Taxonomie	13

15 PSE-Integration	13
15.1 Rolle von PSE	13
15.2 Neue Crystal-Typen	14
16 Implementierungsarchitektur v0.3	14
16.1 Vorgeschlagene Rust-Crates	14
16.2 Kerntraits	15
17 Algorithmischer Gesamtablauf	15
18 Validierungsprotokoll	16
18.1 Tests	16
18.2 Metriken	16
19 Governance, Ethik und Sicherheitsgrenzen	17
20 Produktiver Nutzen	17
20.1 Warum diese Iteration Wert erzeugt	17
20.2 Konkrete Zielprodukte	17
21 Definition of Done für v0.3	18
22 Schlussfolgerung	18
23 ProblemSpec und Domänenadapter	19
23.1 ProblemSpec	19
23.2 Domänenadapter	19
24 Hypercube-Konstruktionsalgorithmus	20
24.1 Von Rohmaterial zu Dimensionen	20
24.2 Wertedomänen	20
24.3 Kopplungen	21
25 Topologische Messschicht	21
25.1 Kopplungsgraph	21
25.2 Zentralitäts- und Schnittmetriken	21
25.3 Singularitäten	21
26 Pfadalgebra	22
26.1 Pfade als erste Klasse	22
26.2 Pfadoperationen	22
27 Carrier-Scheduler	22
27.1 Warum Scheduling notwendig ist	22
28 Norm-Lifecycle und Fitnessökonomie	23
28.1 Normen entstehen aus erfolgreichen Läufen	23
28.2 Promotion, Quarantäne, Deaktivierung	23
29 Adversarielle Robustheit und Counter-Operator	23
29.1 Defensive Lesart	23
29.2 Schutzmechanismen	24

30 Emissions- und Exit-Audit	24
30.1 Emissionen	24
30.2 Exit-Audit	24
31 Anwendungsprofile	24
31.1 Profil A: Software-Synthese	24
31.2 Profil B: Wissenserschliessung	25
31.3 Profil C: Resilienz- und Optionalitätsaudit	25
31.4 Profil D: Prozessplanung	25
32 Roadmap	25
32.1 Phase 1: Spezifikationshärtung	25
32.2 Phase 2: PSE-Anbindung	25
32.3 Phase 3: Agent Harness	26
32.4 Phase 4: Produktisierung	26
33 Masterprompt für Agentenimplementierung	26
A Glossar	26
B Quellenbasis dieser Synthese	27
C Maschinenlesbare Kurzspezifikation	27

1 Executive Summary

Die vorherige Version beschrieb den Hypercube-Decomposer als Puzzle-Solver. Der Hypercube ist ein vorgeformter, aber noch ungelöster Möglichkeitsraum. Constraints, Normen, Beobachtungen und Teilentscheidungen füllen ihn schrittweise. Ab einer kritischen Constraint-Masse kann der verbleibende Lösungsraum durch Propagation und topologische Kopplung kollabieren. Diese Iteration behält diesen Kern und erweitert ihn in drei Richtungen.

- E1. Optionalitätsgeometrie:** Nicht jede formal vorhandene Option ist operativ erreichbar. Ein Hypercube muss daher nicht nur Alternativen, sondern auch Pfade, Pfadkosten, Pfadbrüche und Pfadexzisionen modellieren.
- E2. Carrier-Ladder:** Ein System besitzt nicht nur Dimensionen, sondern Träger. Operative Last kann zwischen Trägern migrieren. Diese Migration ist ein kontrollierter topologischer Übergang, nicht bloss ein Parameterwechsel.
- E3. Anti-Drift:** Systeme driften, wenn ihre innere Krümmung, Kohärenz oder Optionalitätsstruktur zerfällt. Ein Anti-Drift-Operator hält den Systemraum innerhalb eines zulässigen Attraktorgebiets.
- E4. Constraint-Lattice:** Constraints bilden nicht nur eine Liste, sondern einen Verband. Tiefer im Verband bedeutet stärker eingeschränkt. Solve und Coagula bewegen den Zustand entlang dieses Verbandes.
- E5. Dual-Fabric:** Jeder strukturelle Commit sollte eine primäre und eine gespiegelte oder transversale Sicht durchlaufen. Nur wenn beide Sichten konsistent sind, darf der Commit als robust gelten.
- E6. Fail-Closed-Governance:** Scheitern darf nicht in unkontrolliertes Weitergenerieren übergehen. Jeder Gate-Fail erzeugt entweder eine Verfeinerung, eine Boundary, eine Normlücke, eine Migration oder einen sauberen Abbruch.
- E7. Defensive Feldanalyse:** Das Framework wird ausdrücklich nicht als Werkzeug verdeckter Manipulation formuliert. Es dient der Modellierung, Sichtbarmachung, Prüfung, Resilienz und kontrollierten Forschung.

Die wichtigste neue These lautet:

Ein topologischer Traversierungsraum ist nicht nur ein Lösungsraum, sondern ein Raum aus Optionen, Pfaden, Trägern, Constraints, Krümmungen und Evidenz. Der eigentliche Hebel entsteht, wenn diese Ebenen gemeinsam kollabiert werden.

2 Quellenneutralisierung und chirurgische Extraktion

2.1 Methode

Die neuen Ressourcen wurden nicht als Domänenwerke übernommen. Sie wurden strukturell zerlegt. Jedes Werk enthält eine thematische Oberfläche und darunter eine neutralisierbare Mechanik. Für das Framework zählt nur die Mechanik. Der Rest wird verworfen.

Definition 2.1 (Chirurgische Neutralisierung). *Chirurgische Neutralisierung ist die Abbildung*

$$\mathcal{N}_{ext} : W \rightarrow M, \tag{1}$$

wobei ein Werk W in eine Menge domänenneutraler Mechanismen M überführt wird. Ein Mechanismus ist zulässig, wenn er als Graphoperation, Constraint-Regel, Zustandsübergang, Evidenzschema, Operator, Gate oder Invariante formulierbar ist.

2.2 Extraktionstabelle

Quelle	Oberflächendomäne	Neutralisierter Mechanismus
Topologisches Traversierungsframework v0.2	Hypercube-Puzzle, 51-Prozent-Kollaps, Wissenser-schliessung	Blanker Hypercube, Constraint-Propagation, negative Topologie, CrystalSet, Theomimetik als Entwurfsheuristik.
Gekrümmte Welt	Konsensbildung, Optionalitätsverengung, Pfadexzision	Optionalitätsgraph, operative Pfadverfügbarkeit, Path-Excision-Detector, Feldkrümmung, Kompression-Asymmetrie-Verstärkung-Konvergenz als Mor-phismusfamilie.
Phase-Ladder des Dollar-Systems	Geldsystemarchitektur, Carrier-Migration, Anti-Drift	Geordnete Trägerleiter, Lastmigra-tion, Sättigungsschwellen, Friktions-und Schocktrigger, tri-temporale Zu-stands-führung, Anti-Drift als Krüm-mungserhaltung.
The Physics of Mass Control	Operatorischer Feld-Konsens-Kalkül	State Algebra, Operator Registry, Constraint-Lattice, Resonite/Infogene, WEAVE, Dual-Fabric, Stitcher-Mirror, Merkaba-Gate, Emissions-audit, Fail-Closed, adversarielle Ro-bustheit.

2.3 Was ausdrücklich nicht übernommen wird

- keine konkrete politische, monetäre oder gesellschaftliche Zielsetzung;
- keine Anleitung zur Beeinflussung realer Personen oder Gruppen;
- keine Behauptung, jedes System sei exakt als Hypercube lösbar;
- keine Gleichsetzung von Metaphern mit physikalischen Gesetzen;
- keine unbeschränkte 51-Prozent-Regel ohne Domänenkalibrierung;
- keine verdeckte Optimierung gegen menschliche Autonomie.

3 Das Grundmodell: Vom Puzzle-Cube zum Feld-Cube

3.1 Hypercube, Pfadraum und Carrier-Raum

Die vorherige Fassung modellierte den Hypercube als Möglichkeitsraum

$$\Omega = A_1 \times A_2 \times \cdots \times A_n. \quad (2)$$

Dies reicht für kombinatorische Entscheidungen. Für reale Systeme ist es jedoch unvollständig, weil eine Option nicht nur existieren, sondern auch erreichbar sein muss. Daher wird der Hypercube erweitert:

Definition 3.1 (Feld-Cube). *Ein Feld-Cube ist ein Tupel*

$$\mathfrak{C} = (D, A, K, G, P, \Lambda, \kappa, H), \quad (3)$$

wobei D Dimensionen, A Wertedomänen, K Constraints, G Kopplungen, P Pfade, Λ Carrier, κ Krümmungs- oder Kohärenzfunktion und H Evidenz bezeichnet.

Der Feld-Cube ist strenger als der blanke Hypercube. Er fragt nicht nur:

Welche vollständigen Belegungen sind logisch möglich?

sondern auch:

Welche Belegungen sind erreichbar, tragbar, stabil, beweisbar und replaybar?

3.2 Vier Raumebenen

Ebene	Frage	Operatoren
Möglichkeitsraum	Was könnte formal sein?	Dimensionierung, Kartesisches Produkt, Typisierung.
Pfadraum	Was ist operativ erreichbar?	Optionalität, Pfadkosten, Pfadexzision, Boundary Contracts.
Carrier-Raum	Wo liegt operative Last?	Phase-Ladder, Carrier-Migration, Sättigung, Anti-Drift.
Evidenzraum	Was darf committed werden?	Gate, Falsifier, Replay, Crystal, Audit.

4 Optionalitätsgeometrie

4.1 Formale und operative Optionen

Die aus *Gekrümmte Welt* extrahierbare Kernunterscheidung ist die zwischen formaler Möglichkeit und operativer Möglichkeit. Für das Traversierungsframework ist dies fundamental. Ein Hypercube, der nur formale Optionen enthält, kann falsche Freiheitsgrade anzeigen. Er behauptet dann Lösbarkeit, obwohl die Pfade zur Lösung fehlen.

Definition 4.1 (Formale Option). *Eine formale Option ist ein Wert $a \in A_i$, der im Wertbereich einer Dimension enthalten ist und nicht unmittelbar durch Typregeln ausgeschlossen wird.*

Definition 4.2 (Operative Option). *Eine operative Option ist eine formale Option $a \in A_i$, für die mindestens ein zulässiger Pfad*

$$\pi : s_0 \rightsquigarrow (D_i = a) \quad (4)$$

existiert, dessen Kosten, Ressourcen, Vorbedingungen und Risiken unter den Schwellen des Run Descriptors liegen.

Definition 4.3 (Optionalität). *Die Optionalität eines Zustands s ist*

$$\mathcal{O}(s) = \sum_{a \in A_{\text{formal}}(s)} \mathbf{1}[\exists \pi_a \in P_{\text{adm}}(s)] \cdot w(a, \pi_a), \quad (5)$$

wobei w Nutzen, Stabilität, Evidenzwert oder strategische Relevanz gewichten kann.

4.2 Pfadexzision

Definition 4.4 (Pfadexzision). *Pfadexzision liegt vor, wenn eine Option formal im Raum verbleibt, aber alle admittierbaren Pfade zu ihr entfernt oder blockiert sind:*

$$a \in A_{\text{formal}}(s) \quad \wedge \quad \neg \exists \pi_a \in P_{\text{adm}}(s). \quad (6)$$

Im Framework ist Pfadexzision keine moralische Kategorie, sondern ein Diagnoseobjekt. Sie zeigt, dass der Modellraum eine falsche Freiheit enthält. Diese falsche Freiheit muss entweder als Boundary markiert, aus dem Feasible Set entfernt oder durch neue Träger/Pfade wieder operationalisiert werden.

Listing 1: Path-Excision-Detector

```
for each dimension D_i:
  for each formal value a in A_i:
    paths = admissible_paths(current_state, target=(D_i=a))
    if paths.is_empty():
      mark PathExcision(D_i, a)
      reduce operational_optionality
      create GapCrystal(type="path_excision")
```

4.3 Kompression, Asymmetrie, Verstärkung, Konvergenz

Aus der gekrümmten Welt werden vier neutrale Morphismen extrahiert:

Morphismus	Technische Bedeutung	Messbares Signal
Kompression	Reduktion operativer Pfade bei Erhaltung formaler Optionen.	Sinkende Optionalität, steigende Pfadkosten, zunehmende Boundary-Dichte.
Asymmetrie	Ungleichverteilung von Sicht, Kosten, Risiko oder Steuerbarkeit.	Differenz zwischen Agenten-Optionalitäten, Informationsgradienten.
Verstärkung	Kleine Signale werden durch Kanalstruktur überproportional sichtbar.	Amplification Ratio zwischen Quellsignal und Feldwirkung.
Konvergenz	Unabhängige Pfade enden in ähnlichen Attraktoren.	Sinkende Varianz der Endzustände trotz heterogener Startpunkte.

Definition 4.5 (Feldkrümmung). *Die Feldkrümmung eines Zustandsraums ist eine Funktion*

$$\mathcal{K}_{\text{field}}(s) = f(\nabla \mathcal{O}(s), C_{\text{path}}(s), A_{\text{asym}}(s), R_{\text{amp}}(s), V_{\text{conv}}(s)), \quad (7)$$

die misst, wie stark die operative Beweglichkeit in bestimmte Attraktoren gebogen wird.

5 Phase-Ladder und Carrier-Migration

5.1 Carrier als Träger operativer Last

Der Phase-Ladder-Begriff wird aus seiner monetären Domäne neutralisiert. Ein Carrier ist jede Struktur, die Last, Evidenz, Kontext, Entscheidung, Aufmerksamkeit, Liquidität, Rechenaufwand, Risiko oder semantische Kopplung tragen kann.

Definition 5.1 (Carrier). *Ein Carrier ist ein Tupel*

$$\lambda_i = (id_i, cap_i, load_i, phase_i, sat_i, adm_i, role_i), \quad (8)$$

wobei cap_i Kapazität, $load_i$ aktuelle Last, $phase_i$ Phasenlage, sat_i Sättigungsschwelle, adm_i Admissibilitätsbedingungen und $role_i$ Funktionsrolle bezeichnet.

Definition 5.2 (Phase-Ladder). *Eine Phase-Ladder ist eine geordnete Menge phasenversetzter Carrier*

$$\Lambda = \{\lambda_0, \lambda_1, \dots, \lambda_L\}, \quad (9)$$

in der λ_0 die aktive Primärphase und λ_i für $i > 0$ vorbereitete Sekundär-, Reserve-, Brücken- oder Absorptionsphasen darstellen.

5.2 Carrier-Migration

Definition 5.3 (Carrier-Migration). *Carrier-Migration ist ein Zustandsübergang*

$$\text{Migrate}_{a \rightarrow b} : \lambda_a \rightarrow \lambda_b, \quad (10)$$

bei dem operative Last von einem Carrier λ_a auf einen Carrier λ_b übertragen wird, ohne dass die globalen Invarianten des Systems verletzt werden.

Eine Migration wird ausgelöst, wenn mindestens eine Druckbedingung und eine Zielreifebedingung erfüllt sind:

$$(F_a \geq \theta_F \vee S_a \geq \theta_S \vee load_a \geq sat_a) \wedge \text{Ready}(\lambda_b) \geq \theta_R. \quad (11)$$

Dabei bezeichnet F Friktion, S Schock, sat Sättigung und Ready die Reife der Zielphase.

5.3 Anwendungsfälle im Framework

Framework-Domäne	Carrier	Migration
Software-Synthese	Module, Services, Datenmodelle, Testharnesses	Last von Monolith-Design zu Microservice-, Plugin- oder Adapterstruktur.
Wissenserschliessung	Hypothesen, Quellenklassen, Modelle	Wechsel von spekulativer Hypothese zu evidenzdominierter Modellfamilie.
Audit/Compliance	Nachweisarten, Kontrollpunkte, Logs	Migration von narrativem Nachweis zu maschinenprüfbarem Evidence Bundle.
Forschung	Theorien, Messmethoden, Experimente	Migration von Begriffsklärung zu falsifizierbaren Testprotokollen.
Betrieb/Monitoring	Sensoren, Metriken, Alarmkanäle	Migration von Normalbetrieb zu Incident-, Recovery- oder Fail-Closed-Carrier.

5.4 Anti-Drift

Definition 5.4 (Drift). *Drift liegt vor, wenn ein Systemzustand seine Zielinvarianten, Kohärenz, Optionalität oder Replayfähigkeit über Zeit verliert:*

$$\text{Drift}(s_t) = d(s_t, \mathcal{A}) > \epsilon, \quad (12)$$

wobei $\mathcal{A}$ das zulässige Attraktorgebiet bezeichnet.

Definition 5.5 (Anti-Drift-Operator). *Ein Anti-Drift-Operator ist eine Abbildung*

$$\text{AntiDrift} : \Sigma_t \rightarrow \Sigma_{t+1}, \quad (13)$$

die Drift reduziert, ohne die Replay-, Evidence- oder Gate-Invarianten zu verletzen.

Anti-Drift kann durch vier Mechanismen erfolgen:

1. **Constraint-Nachschärfung:** schwache Freiheitsgrade werden stärker gebunden.
2. **Carrier-Migration:** Last wird auf tragfähigere Phase verschoben.
3. **Boundary-Neubildung:** instabile Kopplungen werden als Boundary Contracts explizit gemacht.
4. **Norm-Rekalibrierung:** Normfitness und Scope werden angepasst, wenn historische Normen Drift erzeugen.

Listing 2: Anti-Drift-Kontrollschleife

```
measure drift against attractor A
if drift <= epsilon:
    continue
else:
    candidates = [tighten_constraints, migrate_carrier, form_boundary,
recalibrate_norms]
    for op in ranked(candidates):
        proposal = op(state)
        if Gate(proposal).pass and Replay(proposal).pass:
            Commit(AntiDriftCrystal(proposal))
            break
    if no proposal passes:
        FailClosed("anti-drift unresolved")
```

6 Tri-temporale Architektur

Die Phase-Ladder-Quelle liefert eine wertvolle Zeitarchitektur. Sie wird neutralisiert als drei Zeitachsen:

Zeitform	Bedeutung	Framework-Rolle
Prätemporal	Zustandslose Referenz, Null-Center, leere Ordnung.	Canonical Baseline, Genesis, unverbrauchte Normreferenz.
Intrinsisch	Kontinuierliche interne Dynamik.	Solver-Iteration, Carrier-Phase, Resonanz, Driftmessung.

Invariante 6.1 (Tri-temporale Trennung). *Ein Framework-Lauf MUSS prätemporale Referenz, intrinsische Iteration und extrinsischen Commit getrennt halten. Eine Änderung im intrinsischen Zustand darf erst dann extrinsisch sichtbar werden, wenn Gate, Evidence und Replay bestanden wurden.*

7 Constraint-Lattice, Resonite, Infogene

7.1 Resonite als messbarer Constraint-Kern

Die frühere Fassung enthielt Resonite bereits als kleinste strukturelle Beobachtung. Die neue Iteration verschärft: Ein Resonite ist nicht bloss ein Merkmal, sondern ein messbarer Constraint-Kern.

Definition 7.1 (Resonite). *Ein Resonite ist eine messbare Abbildung*

$$g : X \rightarrow \mathbb{R}^m, \quad (14)$$

die eine strukturelle Eigenschaft eines Zustands, Artefakts, Pfads oder Carriers extrahiert und als Constraint-Kandidat verwendbar macht.

Beispiele: Kopplungsgrad, Pfadkosten, Spektrallücke, Kohärenz, Testabdeckung, Quellendichte, Drift, Optionalität, Unsicherheitsmasse, MCI, Sättigung.

7.2 Constraint-Lattice

Definition 7.2 (Constraint-Lattice). *Ein Constraint-Lattice ist ein Verband*

$$\mathcal{L} = (\mathcal{A}, \subseteq, \cap, \cup), \quad (15)$$

wobei jedes Element $A \in \mathcal{A}$ eine admissible region im Möglichkeitsraum bezeichnet. Tiefer im Lattice bedeutet stärker eingeschränkt.

Definition 7.3 (Solve-Filter). *Für eine Resonite-Familie $S = \{g_1, \dots, g_m\}$ und Schwellen θ_i ist der Solve-Filter*

$$F_S(X) = \{x \in X \mid \forall i : g_i(x) \geq \theta_i\}. \quad (16)$$

7.3 Infogene und Infogenome

Definition 7.4 (Infogen). *Ein Infogen ist ein ausführbares Constraint-Fragment*

$$\iota = (g, \theta, \text{scope}, \text{fitness}, \text{evidence}, \text{status}), \quad (17)$$

das eine Resonite-Messung in eine operative Einschränkung überführt.

Definition 7.5 (Infogenom). *Ein Infogenom ist eine geordnete Menge von Infogenen*

$$I = (\iota_1, \dots, \iota_k), \quad (18)$$

die zusammen ein lösungsraumkontrahierendes Programm bilden.

8 WEAVE: Deterministische Key-Space-Scherung

WEAVE wird neutralisiert als deterministisches Verfahren zur Ordnungsbildung in Constraint-Räumen. Es sortiert nicht semantisch nach Geschmack, sondern nach reproduzierbarer Hebelwirkung.

Definition 8.1 (WEAVE-Priorität). *Für einen Constraint K_j ist die WEAVE-Priorität*

$$q(K_j) = \alpha \Delta H_j + \beta \text{centrality}_j + \gamma \text{evidence}_j - \delta \text{cost}_j, \quad (19)$$

wobei ΔH_j erwartete Entropiereduktion, centrality_j Graphzentralität, evidence_j Evidenzstärke und cost_j Anwendungskosten bezeichnet.

Listing 3: WEAVE-Constraint-Ordering

```
constraints = collect_candidate_constraints(state)
for K in constraints:
    K.score = alpha * entropy_reduction(K)
            + beta * graph_centrality(K)
            + gamma * evidence_strength(K)
            - delta * application_cost(K)
ordered = sort_descending(constraints, key=score)
apply ordered constraints until gate, budget or collapse threshold
```

9 Dual-Fabric und Stitcher-Mirror

9.1 Warum eine einzige Sicht nicht reicht

Ein topologischer Commit kann lokal plausibel, aber global falsch sein. Darum benötigt das Framework eine Dual-Fabric-Prüfung. Jede Kandidatenstruktur wird durch zwei Stoffe geführt:

- P^+ : primäre, konstruktive oder emissionsseitige Sicht;
- P^- : spiegelnde, inverse, adversarielle oder feldseitige Sicht.

Definition 9.1 (Dual-Fabric State). *Ein Dual-Fabric-State ist*

$$X^\pm = (x^+, x^-, \sigma^+, \sigma^-, MCI), \quad (20)$$

wobei x^+, x^- Zustände, σ^+, σ^- Signaturen und MCI der Mirror-Consistency-Index sind.

Definition 9.2 (Mirror-Consistency-Index). *Der Mirror-Consistency-Index ist*

$$MCI = 1 - \text{dist}(\sigma^+, \sigma^-), \quad (21)$$

normalisiert auf $[0, 1]$. Ein Commit ist nur zulässig, wenn $MCI \geq \eta$.

9.2 Stitcher und Mirror

Operator	Funktion	Failure Mode
Stitcher	Fügt Teilzustände über Boundaries zu kohärentem Kandidaten.	Nahtbruch, inkonsistente Randbedingungen.

Mirror	Erzeugt inverse, duale oder adversarielle Sicht.	Asymmetrie bleibt unentdeckt.
MCI-Gate	Prüft, ob Primär- und Spiegelsicht hinreichend konsistent sind.	Falscher Commit bei lokaler Plausibilität.

10 Der erweiterte Solve-Gate-Coagula-Zyklus

10.1 Nukleus

Der operative Kern bleibt:

$$T = \text{Coagula} \circ \text{Gate} \circ \text{Solve}. \quad (22)$$

In v0.3 wird dieser Kern erweitert:

$$T_{v0.3} = \text{Commit} \circ \text{Verify} \circ \text{Gate}_{\text{dual}} \circ \text{AntiDrift} \circ \text{Migrate} \circ \text{Coagula} \circ \text{Solve} \circ \text{Project}. \quad (23)$$

10.2 Projektionsphase

$$\text{Project}_v(\Sigma) = \text{Base}(\Sigma) \parallel \text{Focus}_v(\Sigma) \parallel \text{Norms}_v \parallel \text{Paths}_v \parallel \text{Carriers}_v \parallel \text{Evidence}_v. \quad (24)$$

Diese Projektion ist der Chameleon-Kern. Sie entscheidet, was ein Solver sehen darf und muss.

10.3 Solve

Solve erzeugt Kandidaten:

$$\text{Cand} = \text{Solve}(\text{Project}_v(\Sigma), RD). \quad (25)$$

Zulässige Solver:

- deterministische Regeln;
- Constraint-Solver;
- Templates und Blueprints;
- PSE-Crystal-Query;
- LLM/Oracle;
- menschliche Entscheidung;
- hybride Kombinationen.

10.4 Gate

Gate prüft:

1. Typ- und Schema-Korrektheit;
2. harte Constraints;
3. Pfadverfügbarkeit;
4. Carrier-Reife;

5. Normkonflikte;
6. Dual-Fabric-MCI;
7. Falsifizierbarkeit;
8. Replayfähigkeit;
9. Evidence-Vollständigkeit;
10. Anti-Stealth- und Governance-Kriterien.

10.5 Coagula

Coagula transformiert Scheitern in Struktur:

Failure	Coagula-Aktion
Constraint-Fail	Constraint explizit machen, Scope verschärfen, Kandidat verwerfen.
Path-Fail	Pfadexzision markieren, Boundary bilden oder Alternative suchen.
Carrier-Fail	Migration prüfen, Reservecarrier aktivieren oder Last reduzieren.
MCI-Fail	Dual-Fabric weiter zerlegen oder Mirror-Sicht als Genevidenz committen.
Falsifier-Fail	Hypothese zurückstufen, Surrogatklasse erweitern, Knowledge Gap erzeugen.
Replay-Fail	Nicht committen; deterministische Reproduktion reparieren.
Budget-Fail	Fail-Closed; offenen Zustand als GapCrystal speichern.

11 51-Prozent-Kollaps als Constraint-Perkolation

11.1 Warum die Zahl nicht naiv gelesen werden darf

Die 51-Prozent-Grenze ist keine universelle mathematische Wahrheit. Ein Constraint kann trivial sein oder den gesamten Raum halbieren. Entscheidend ist daher gewichtete strukturelle Masse, nicht Anzahl.

Definition 11.1 (Constraint-Masse). *Die Masse eines Constraints ist*

$$m(K_j) = \Delta H(K_j) \cdot \text{centrality}(K_j) \cdot \text{evidence}(K_j) \cdot \text{stability}(K_j). \quad (26)$$

Definition 11.2 (Constraint-Coverage). *Die gewichtete Coverage einer Teilbelegung ist*

$$\Gamma(\phi) = \frac{\sum_{K_j \in K_{\text{active}}(\phi)} m(K_j)}{\sum_{K_j \in K_{\text{critical}}} m(K_j)}. \quad (27)$$

Definition 11.3 (Perkolationsschwelle). *Eine Domäne besitzt eine Perkolationsschwelle p_c , wenn für $\Gamma(\phi) > p_c$ die erwartete Zahl strukturell nichtäquivalenter Komplettierungen unter eine definierte Grenze fällt:*

$$\mathbb{E}[|F_K(\phi) / \sim|] \leq \tau_{\text{sol}}. \quad (28)$$

11.2 Collapse Certificate

Definition 11.4 (Collapse Certificate). *Ein Collapse Certificate ist ein Nachweis*

$$CC = (\Gamma, \Delta H, |F|, |F| \sim |, MCI, p_{fals}, replay, hash), \quad (29)$$

der zeigt, dass der Kollaps nicht bloss behauptet, sondern operational erreicht wurde.

12 Negative Topologie und Wissenserschliessung

12.1 Unwissen als Form

Unwissen ist nicht nur fehlender Inhalt. Es hat Topologie. Es gibt isolierte Lücken, Schleifen, Sackgassen, fehlende Brücken und verborgene Pfade.

Definition 12.1 (Knowledge Gap). *Ein Knowledge Gap ist ein Bereich $U \subset \Omega$, der für die Zielentscheidung relevant ist, aber unter aktueller Evidenz nicht eindeutig belegt, ausgeschlossen oder traversiert werden kann.*

Definition 12.2 (Negative Topologie). *Negative Topologie ist die Struktur der fehlenden, blockierten oder unbewiesenen Relationen im Modellraum:*

$$\mathcal{T}^- = (G_{expected} \setminus G_{observed}) \cup PathExcision \cup EvidenceGaps. \quad (30)$$

12.2 Betti-Lücken

Signal	Topologische Lesart	Aktion
β_0 hoch	Viele getrennte Komponenten.	Brücken suchen, Cluster separat lösen.
β_1 hoch	Viele Zyklen oder unaufgelöste Schleifen.	Widersprüche, zirkuläre Abhängigkeiten, Boundary Contracts prüfen.
β_2 hoch	Höherdimensionale Hohlräume.	Fehlende Theorie, fehlendes Modell, nicht beobachteter Träger.
Sinkende Spektrallücke	Schwache Kopplung oder fragile Kohärenz.	Carrier-Migration oder Constraint-Nachschärfung.

13 Multi-Solution-Handling

Nicht alle Cubes kollabieren auf einen eindeutigen Zustand. Viele reale Räume besitzen mehrere gültige Lösungen.

Definition 13.1 (Solution Set). *Ein Solution Set ist*

$$S^* = \{x \in F_K \mid Gate(x) = 1 \wedge Replay(x) = 1\}. \quad (31)$$

Definition 13.2 (CrystalSet). *Ein CrystalSet ist ein content-addressed Container*

$$CS = (id, \{C_1, \dots, C_m\}, rank, pareto, selection_rule, evidence). \quad (32)$$

Lösungstyp	Behandlung
Eindeutig	Einzelner Crystal.

Quasi-eindeutig	Einzelner Crystal plus Äquivalenzklasse.
Pareto-Front	CrystalSet mit Trade-off-Achsen.
Archetypen	Mehrere Pattern-Crystals mit Nutzungsbedingungen.
Offene Mannigfaltigkeit	Kein Commit als Lösung; GapCrystal plus Explorationsplan.

14 Operator Registry und Status-Taxonomie

14.1 Operator Registry

Jeder Operator muss registriert werden:

Listing 4: Operator-Registry-Schema

```
Operator := {
  id,
  version,
  input_type,
  output_type,
  determinism_class: Pure | Boundary | Oracle | Human,
  admissibility_predicate,
  evidence_schema,
  failure_modes,
  replay_requirements,
  status: Defined | Derived | ModelSpecific | External | Open
}
```

14.2 Status-Taxonomie

Status	Bedeutung
Defined	Begriff oder Operator ist innerhalb der Spezifikation definiert.
Derived	Aussage folgt aus Definitionen unter genannten Annahmen.
Model-Specific	Aussage gilt für die gewählte Implementierung oder Domäne.
External	Aussage hängt von externen Fakten, Daten oder Messungen ab.
Open	Aussage ist Forschungsfrage oder Implementierungslücke.

15 PSE-Integration

15.1 Rolle von PSE

PSE bleibt der Commit- und Evidence-Kern. Das Traversierungsframework erzeugt Kandidaten, Pfade, Carrier-Zustände und Constraint-Collapses. PSE entscheidet, welche strukturellen Ereignisse als Crystal committed werden können.

PSE-Komponente	Rolle in TTF v0.3
Observation	Eingang für Stream-, Artefakt- und Prozessereignisse.
PersistentGraph	Projektion der Beobachtungen in einen topologischen Zustand.
5D Event State	Dynamisches Profil von Potential, Dichte, Frequenz, Konnektivität, Kausalität.

Kairos Gate	Achtfacher Konjunktionsfilter für Commit-Reife.
Falsifier	Nullmodellprüfung gegen Surrogatströme.
SemanticCrystal	Content-addressed, replaybarer Strukturcommit.
Operatoralgebra	Compose, Dual, Bridge, Query, Interpolate für Crystal-Verarbeitung.
Pattern Memory	Wiedererkennung bereits bekannter Strukturformen.

15.2 Neue Crystal-Typen

Listing 5: Neue Crystal-Spezialisierungen

```

PathExcisionCrystal := Crystal + {
    dimension,
    formal_option,
    missing_paths,
    operational_impact
}

CarrierMigrationCrystal := Crystal + {
    source_carrier,
    target_carrier,
    trigger_metrics,
    load_delta,
    post_migration_stability
}

AntiDriftCrystal := Crystal + {
    drift_measure,
    correction_operator,
    attractor_reference,
    residual_drift
}

GapCrystal := Crystal + {
    gap_type,
    missing_evidence,
    exploration_plan,
    risk_of_false_closure
}

CrystalSet := Crystal + {
    members,
    equivalence_relation,
    pareto_axes,
    selection_rule
}

```

16 Implementierungsarchitektur v0.3

16.1 Vorgeschlagene Rust-Crates

Crate	Aufgabe
-------	---------

ttf-types	Gemeinsame Typen: FeldCube, Optionality, Carrier, Path, CollapseCertificate.
ttf-dof	Freiheitsgradgraph, Couplings, Singularitäten, Constraint-Masse.
ttf-optionality	Optionalitätsmessung, Pfadkosten, Pfadexzision.
ttf-carrier	Phase-Ladder, Carrier-Migration, Sättigung, Anti-Drift.
ttf-lattice	Constraint-Lattice, Resonite, Infogene, WEAVE.
ttf-dual	Dual-Fabric, Stitcher-Mirror, MCI.
ttf-solver	Solver-Trait, Oracle-Adapter, Templates, deterministic solvers.
ttf-gate	Gate-Komposition, Failure-Typen, Fail-Closed-Policy.
ttf-crystal	Spezialisierte Crystal-Wrapper und CrystalSet.
ttf-replay	Traversal replay, deterministic run verification.
ttf-pse-bridge	Integration mit PSE: Observation, macro-step, Crystal registry.

16.2 Kerntraits

Listing 6: Trait-Skizze

```

trait Solver {
  type Input;
  type Candidate;
  fn solve(&self, input: Self::Input, rd: &RunDescriptor) -> Vec<Self::Candidate>;
}

trait Gate<Candidate> {
  fn evaluate(&self, candidate: &Candidate, state: &State) -> GateResult;
}

trait CarrierPolicy {
  fn ready(&self, carrier: &Carrier, state: &State) -> f64;
  fn migrate(&self, from: CarrierId, to: CarrierId, state: &mut State) ->
  MigrationProof;
}

trait OptionalityModel {
  fn formal_options(&self, state: &State) -> Vec<OptionValue>;
  fn admissible_paths(&self, option: &OptionValue, state: &State) -> Vec<Path>;
}

trait AntiDriftPolicy {
  fn drift(&self, state: &State, attractor: &Attractor) -> f64;
  fn correct(&self, state: &State) -> Vec<CorrectionCandidate>;
}

```

17 Algorithmischer Gesamt Ablauf

Listing 7: TTF v0.3 End-to-End-Run

```

input: ProblemSpec, Corpus, RunDescriptor RD

```

```

1. Canonicalize inputs and build initial FieldCube C
2. Extract resonites, constraints, norms and candidate carriers
3. Build optionality graph and detect path excisions
4. Build phase ladder Lambda and assign load to primary carrier
5. Compute coupling graph, Betti gaps, spectral signals and singularities
6. Generate CollapsePlan using WEAVE priority and Fiedler/Kuramoto grouping
7. For each traversal node v:
    projection = Project(state, v)
    candidates = Solve(projection)
    for candidate in candidates:
        candidate = optional CarrierMigration(candidate)
        candidate = optional AntiDrift(candidate)
        gate = DualFabricGate(candidate)
        if gate.pass:
            proof = Verify(candidate)
            if proof.replay && proof.evidence_complete:
                crystal = Commit(candidate, proof)
                update norms, registry, pattern memory
                break
    if no candidate passed:
        Coagula(failure)
        if unrecoverable: FailClosed
8. If multiple valid solutions remain: emit CrystalSet
9. Produce Manifest, Replay Bundle, Gap Report and Implementation Plan

```

18 Validierungsprotokoll

18.1 Tests

Test	Kriterium	Pflicht
Determinismus	Gleicher RD und gleiche Inputs erzeugen identische Crystal-IDs.	Muss
Path-Excision	Formal vorhandene, aber unerreichbare Optionen werden erkannt.	Muss
Carrier-Migration	Migration erfolgt nur bei Trigger + Zielreife.	Muss
Anti-Drift	Driftkorrektur verletzt keine Gate- oder Replay-Invariante.	Muss
MCI	Duale Sicht muss konsistent genug sein.	Muss
Falsifier	Empirische Strukturbehauptungen werden gegen Nullmodell geprüft.	Soll
CrystalSet	Multi-Lösung wird nicht künstlich auf Einzellösung reduziert.	Muss
Fail-Closed	Budget-, Gate- oder Replay-Fail erzeugt keinen unbewiesenen Commit.	Muss

18.2 Metriken

$$\text{OptionDensity} = \frac{|A_{\text{operative}}|}{|A_{\text{formal}}|}, \quad (33)$$

$$\text{PathIntegrity} = 1 - \frac{|PathExcision|}{|A_{\text{formal}}|}, \quad (34)$$

$$\text{CarrierPressure}(\lambda_i) = \frac{\text{load}_i}{\text{cap}_i}, \quad (35)$$

$$\text{DriftIndex} = d(s_t, \mathcal{A}), \quad (36)$$

$$\text{CollapseCoverage} = \Gamma(\phi), \quad (37)$$

$$\text{ReplayScore} = \mathbf{1}[\text{Replay}(RD, H) = H]. \quad (38)$$

19 Governance, Ethik und Sicherheitsgrenzen

Dieses Framework analysiert und operationalisiert Strukturmechaniken. Es darf nicht als verdecktes Manipulationssystem gegen reale Personen, Gruppen, demokratische Prozesse oder institutionelle Entscheidungsstrukturen eingesetzt werden. Die neutralisierten Mechanismen aus Feld-, Konsens- und Optionalitätswerken werden hier ausdrücklich defensiv, analytisch und forschungsorientiert verwendet.

Invariante 19.1 (Anti-Stealth). *Jeder externe Einsatz des Frameworks MUSS eine nachvollziehbare Zweckdeklaration, Evidence Chain, Auditierbarkeit und Exit-Bedingung besitzen. Ein Run, dessen Zweck oder Emission nicht deklariert werden kann, darf nicht als konform gelten.*

Invariante 19.2 (Fail-Closed). *Wenn Gate, Replay, Evidence, MCI oder Falsifier fehlschlagen, MUSS der Run entweder verfeinert, als Gap committed oder sauber abgebrochen werden. Er darf nicht in einen unbewiesenen Commit übergehen.*

20 Produktiver Nutzen

20.1 Warum diese Iteration Wert erzeugt

Die neue Schicht macht das Framework nutzbarer, weil sie drei bisher fehlende Realitätsbedingungen einführt:

1. **Erreichbarkeit:** Lösungen sind nicht nur Werte im Cube, sondern Pfade im Feld.
2. **Tragfähigkeit:** Systeme brauchen Carrier; ein einzelner Träger kann saturieren.
3. **Stabilität:** Ein Ergebnis kann korrekt wirken und trotzdem driften; Anti-Drift macht Stabilität explizit.

Damit wird das Framework von einem guten Decomposer zu einer vollständigeren Strukturmaschine.

20.2 Konkrete Zielprodukte

Produkt	Beschreibung
TTF Analyzer	Analysiert vorhandene Systeme auf Freiheitsgrade, Pfade, Pfadexzision, Carrier und Drift.
TTF Planner	Erstellt Collapse-Pläne für komplexe Projekte oder Forschungsvorhaben.
TTF Crystalizer	Committed geprüfte Strukturereignisse als replaybare Crystals.
TTF Knowledge Engine	Modelliert Wissenslücken, Quellenräume und Hypothesenpfade.

TTF Resilience Auditor	Prüft Systeme auf Optionalitätsverlust, Carrier-Sättigung und Anti-Drift-Bedarf.
TTF Agent Harness	Gibt LLMs und Agenten topologisch geschnittene Projektionen statt rohen Prompt-Kontext.

21 Definition of Done für v0.3

Eine Implementierung dieser Spezifikation gilt als v0.3-konform, wenn sie folgende Punkte erfüllt:

- DOD-1.** Sie kann einen ProblemSpec in einen Feld-Cube mit Dimensionen, Constraints, Pfaden und Carriern überführen.
- DOD-2.** Sie kann formale von operativen Optionen unterscheiden.
- DOD-3.** Sie kann Pfadexzisionen erkennen und als Gap oder Crystal erfassen.
- DOD-4.** Sie kann eine Phase-Ladder aufbauen und Carrier-Migrationen regelgebunden ausführen.
- DOD-5.** Sie kann Drift messen und Anti-Drift-Korrekturen gated anwenden.
- DOD-6.** Sie besitzt ein Constraint-Lattice mit WEAVE-Ordering.
- DOD-7.** Sie führt Kandidaten durch Dual-Fabric-Gating.
- DOD-8.** Sie erzeugt Collapse Certificates für deterministische oder quasi-deterministische Kollapses.
- DOD-9.** Sie behandelt Multi-Lösungsräume als CrystalSet statt sie künstlich zu verengen.
- DOD-10.** Sie ist replaybar und erzeugt content-addressed Evidence-Artefakte.

22 Schlussfolgerung

Die rekursive Iteration v0.3 hebt das Topologische Traversierungsframework auf eine höhere Abstraktionsebene. Der Hypercube bleibt der Kern, aber er ist nicht mehr nur ein kombinatorischer Container. Er wird zu einem Feld-Cube: einem Raum aus Möglichkeiten, Pfaden, Carriern, Constraints, Krümmungen, Evidenz und Commit-Regeln. Dadurch wird die Maschine deutlich realistischer. Reale Systeme scheitern selten daran, dass eine Option formal fehlt. Sie scheitern daran, dass Pfade fehlen, Träger saturieren, Constraints falsch gewichtet werden, Drift unbemerkt bleibt oder lokale Lösungen global inkonsistent sind.

Die neuen Bausteine lösen genau diese Lücken. Optionalitätsgeometrie macht Pfade sichtbar. Phase-Ladder macht Träger sichtbar. Anti-Drift macht Stabilität sichtbar. Constraint-Lattice macht Einschränkung sichtbar. Dual-Fabric macht Gegenperspektiven sichtbar. PSE macht den Commit beweisbar.

Damit entsteht eine post-symbolische Strukturmaschine mit hohem Hebel: Sie kann nicht nur erzeugen, sondern Räume erschliessen, Lücken kartieren, Optionen prüfen, Träger migrieren, Lösungen stabilisieren und Beweise speichern. Das ist der nächste kohärente Schritt des Paradigmas.

23 ProblemSpec und Domänenadapter

23.1 ProblemSpec

Damit das Framework nicht als lose Methodensammlung endet, benötigt es einen normierten Eingangstyp. Der Eingang ist nicht ein Prompt, sondern eine strukturierte ProblemSpec. Ein Prompt darf eine ProblemSpec erzeugen, aber er ersetzt sie nicht.

Listing 8: ProblemSpec-Schema

```
ProblemSpec := {
  id,
  title,
  domain,
  objective,
  non_goals,
  input_artifacts,
  observation_sources,
  desired_outputs,
  hard_constraints,
  soft_constraints,
  risk_constraints,
  solver_permissions,
  evidence_requirements,
  falsification_requirements,
  replay_policy,
  emission_policy,
  budget
}
```

Invariante 23.1 (Prompt-Unterordnung). *Ein natürlicher Sprachprompt DARF als Rohmaterial dienen, aber eine konforme Implementierung MUSS ihn zuerst in eine ProblemSpec überführen. Alle späteren Operatoren arbeiten auf der ProblemSpec, nicht auf der freien Promptoberfläche.*

23.2 Domänenadapter

Ein Domänenadapter übersetzt Rohmaterial in Resonite, Observations, Constraints, Pfade und Carrier. Der Kern des Frameworks bleibt unverändert.

Definition 23.1 (Domänenadapter). *Ein Domänenadapter ist ein Tupel*

$$A_D = (\text{Canon}, \text{Extract}, \text{Project}, \text{Verify}, \text{Describe}), \quad (39)$$

wobei *Canon* kanonisiert, *Extract* Struktureinheiten extrahiert, *Project* Domänenmaterial auf TTF-Typen projiziert, *Verify* domänenspezifische Korrektheit prüft und *Describe* menschenlesbare Erläuterung erzeugt.

Adapterklasse	Input	Extraktion
CodeAdapter	Repository, AST, Tests	Resonite aus Typen, Funktionen, Abhängigkeiten, Testsignaturen.
ResearchAdapter	PDFs, Notizen, Quellen	Claims, Evidenzen, Gegenargumente, offene Variablen.
ProcessAdapter	SOPs, Logs, Rollen	Schritte, Engpässe, Pfade, Verantwortlichkeiten, Risiken.

ComplianceAdapter	Normen, Policies, Nachweise	Pflichten, Controls, Evidence Items, Audit-Lücken.
TelemetryAdapter	Zeitreihen, Sensoren, Events	Regime, Anomalien, Carrierdruck, Drift, Falsifier-Targets.

24 Hypercube-Konstruktionsalgorithmus

24.1 Von Rohmaterial zu Dimensionen

Die wichtigste praktische Frage lautet: Woher kommen die Dimensionen des Cubes? Sie entstehen nicht willkürlich. Eine Dimension wird nur zugelassen, wenn sie mindestens eines der folgenden Kriterien erfüllt:

1. Sie entspricht einer wiederkehrenden Entscheidung im Material.
2. Sie trennt zwei beobachtbar unterschiedliche Zustandsklassen.
3. Sie kontrolliert einen Pfad, eine Boundary oder einen Carrier.
4. Sie reduziert messbar Entropie im Lösungsraum.
5. Sie ist für Gate, Replay, Evidenz oder Risiko relevant.

Listing 9: Dimension Discovery

```
raw_units = adapter.extract_units(corpus)
resonites = extract_resonites(raw_units)
candidate_dimensions = []
for r in resonites:
    if recurring_decision(r) or separates_state_classes(r) or controls_path(r):
        candidate_dimensions.push(make_dimension(r))
merge near-duplicates by topology_signature
rank by entropy_reduction and centrality
return selected_dimensions under budget
```

24.2 Wertedomänen

Jede Dimension D_i benötigt eine Wertedomäne A_i . Diese kann diskret, ordinal, kontinuierlich, hierarchisch oder latent sein.

Domänentyp	Beispiel	Behandlung
Diskret	Datenbank = SQLite/Postgres/None	Vollständig enumerierbar.
Ordinal	Risiko = niedrig/mittel/hoch	Ordnung nutzbar für Propagation.
Kontinuierlich	Schwelle $\theta \in [0, 1]$	Diskretisierung oder Intervallarithmetik.
Hierarchisch Latent	Architekturklasse/Subklasse Semantischer Stil, Forschungsrichtung	Baum- oder Lattice-Projektion. Nur über Messprofile und Validatoren zulässig.

24.3 Kopplungen

Kopplungen entstehen, wenn die Wahl einer Dimension die zulässigen Werte anderer Dimensionen verändert.

Definition 24.1 (Kopplungsstärke). *Die Kopplungsstärke zwischen D_i und D_j ist*

$$\omega_{ij} = \mathbb{E}_{a \in A_i} \left[1 - \frac{|A_j \mid D_i = a|}{|A_j|} \right], \quad (40)$$

also die erwartete relative Reduktion der Kandidatenmenge von D_j , wenn D_i belegt wird.

25 Topologische Messschicht

25.1 Kopplungsgraph

Aus Dimensionen und Kopplungen entsteht ein gewichteter Graph

$$G_D = (V_D, E_D, w), \quad V_D = D, \quad (41)$$

wobei eine Kante (D_i, D_j) existiert, wenn $\omega_{ij} > 0$. Dieser Graph ist der operative Kern des Decomposers.

25.2 Zentralitäts- und Schnittmetriken

Metrik	Bedeutung	Einsatz
Degree Centrality	Anzahl direkter Kopplungen.	Frühe Heuristik für Singularitäten.
Betweenness	Vermittlung zwischen Clustern.	Boundary- und Bridge-Erkennung.
Spectral Gap	Robustheit der Kopplung.	Drift- und Fragilitätsmessung.
Fiedler Vector	Natürlicher Graphschnitt.	Dekomposition in Subprobleme.
Clustering Coefficient	Lokale Geschlossenheit.	Norm- und Pattern-Erkennung.
Modularity	Clusterqualität.	Traversierungsplanung.

25.3 Singularitäten

Definition 25.1 (Collapse-Singularität). *Eine Dimension D_i ist eine Collapse-Singularität, wenn ihre Belegung eine überproportionale Reduktion des Lösungsraums erzeugt:*

$$\frac{\Delta H(D_i)}{\text{cost}(D_i)} \geq \theta_{\text{sing}} \quad \wedge \quad \text{centrality}(D_i) \geq \theta_c. \quad (42)$$

Singularitäten sind keine mystischen Punkte. Sie sind strukturelle Hebelpunkte. Ein Solver, der sie ignoriert, verschwendet Budget.

26 Pfadalgebra

26.1 Pfade als erste Klasse

Ein Pfad ist nicht nur eine Reihenfolge von Schritten. Er ist eine beweisbare Möglichkeit, einen Zustand zu erreichen.

Definition 26.1 (Pfad). *Ein Pfad ist ein Tupel*

$$\pi = (s_0, s_1, \dots, s_k; ops; cost; risk; evidence), \quad (43)$$

wobei jeder Übergang $s_t \rightarrow s_{t+1}$ durch einen registrierten Operator erklärt werden muss.

Definition 26.2 (Admittierbarer Pfad). *Ein Pfad ist admittierbar, wenn*

$$cost(\pi) \leq B_{cost} \quad \wedge \quad risk(\pi) \leq B_{risk} \quad \wedge \quad Gate(\pi) = 1. \quad (44)$$

26.2 Pfadoperationen

Operation	Notation	Bedeutung
Komposition	$\pi_1 \circ \pi_2$	Zwei kompatible Pfade werden sequenziell verbunden.
Parallelisierung	$\pi_1 \parallel \pi_2$	Zwei unabhängige Pfade werden gleichzeitig traversiert.
Schnitt	$\pi_1 \cap \pi_2$	Gemeinsamer Teil zweier Pfade.
Dual	π^*	Gegen-, Spiegel- oder Rückwärtssicht eines Pfades.
Exzision	$\pi = \emptyset$	Pfad ist formal erwartet, aber operativ nicht vorhanden.

27 Carrier-Scheduler

27.1 Warum Scheduling notwendig ist

Carrier-Migration darf nicht reaktiv-chaotisch erfolgen. Ein Scheduler entscheidet, wann Last auf welchem Carrier liegt. Er verhindert Überlast, Oszillation und Drift.

Definition 27.1 (Carrier Pressure). *Der Druck eines Carriers ist*

$$P(\lambda_i) = \alpha \frac{load_i}{cap_i} + \beta F_i + \gamma S_i - \delta Ready_i, \quad (45)$$

wobei F_i Friktion, S_i Schock und $Ready_i$ Reife bezeichnet.

Definition 27.2 (Migration Score). *Die Eignung einer Migration $a \rightarrow b$ ist*

$$M_{a \rightarrow b} = P(\lambda_a) \cdot Ready(\lambda_b) \cdot Compat(\lambda_a, \lambda_b) - Cost(a, b). \quad (46)$$

Listing 10: Carrier-Scheduler

```

for each carrier lambda_a:
    pressure = carrier_pressure(lambda_a)
    if pressure > theta_pressure:
        targets = compatible_targets(lambda_a)
        rank targets by migration_score(a,b)

```

```

for lambda_b in targets:
    proof = simulate_migration(a,b)
    if Gate(proof).pass:
        commit CarrierMigrationCrystal
        apply migration
        break

```

28 Norm-Lifecycle und Fitnessökonomie

28.1 Normen entstehen aus erfolgreichen Läufen

Eine Norm darf nicht frei erfunden werden. Sie wird aus validierten Crystals, wiederholten Patterns oder extern deklarierten Anforderungen gebildet.

Definition 28.1 (Norm-Kandidat). *Ein Norm-Kandidat ist*

$$\nu = (\text{predicate}, \text{scope}, \text{evidence}, \text{fitness}, \text{conflicts}, \text{status}). \quad (47)$$

Definition 28.2 (Norm-Fitness). *Die Fitness einer Norm wird rekursiv aktualisiert:*

$$\text{fit}_{t+1}(\nu) = \alpha \text{fit}_t(\nu) + (1 - \alpha) \text{reward}_t(\nu) - \text{penalty}_t(\nu). \quad (48)$$

28.2 Promotion, Quarantäne, Deaktivierung

Status	Bedingung	Aktion
Candidate	Norm aus einem oder wenigen Läufen extrahiert.	Nur lokal verwenden.
Promoted	Wiederholt konfliktfrei erfolgreich.	Scope erweitern.
Quarantined	Widerspruch oder Driftverdacht.	Nur mit Warnung und enger Scope.
Deprecated	Schlechte Fitness oder bessere Norm vorhanden.	Nicht löschen, aber deaktivieren.
Mandatory	Extern oder intern zwingend.	Gate-Fail bei Verletzung.

29 Adversarielle Robustheit und Counter-Operator

29.1 Defensive Lesart

Die neutralisierte Counter-Operator-Schicht ist für Resilienz notwendig. Jeder komplexe Traversierungsraum kann durch falsche Daten, falsche Constraints, manipulierte Pfade, überlastete Carrier oder täuschende Solver-Ausgaben gestört werden.

Definition 29.1 (Counter-Operator). *Ein Counter-Operator ist eine Störung*

$$\tilde{O} : \Sigma \rightarrow \Sigma', \quad (49)$$

die Optionalität, Gate-Entscheidung, Evidenz, Carrierdruck oder Solver-Ausgabe so verändert, dass ein falscher Commit wahrscheinlicher wird.

29.2 Schutzmechanismen

Mechanismus	Schutzwirkung
Dual-Fabric	Gegenprüfung gegen spiegelnde Sicht.
Falsifier	Test gegen Nullmodelle und Surrogate.
Evidence Chain	Nachvollziehbarkeit und Tamper-Erkennung.
Replay	Erkennung nichtdeterministischer oder manipulativ instabiler Outputs.
Carrier-Diversität	Kein Single-Point-of-Failure in Trägerstruktur.
Norm-Quarantäne	Verhindert Ausbreitung schlechter Regeln.
Fail-Closed	Verhindert unbewiesene Emission.

30 Emissions- und Exit-Audit

30.1 Emissionen

Eine Emission ist jedes Artefakt, das den internen Traversierungsraum verlässt: Dokument, Code, Entscheidung, Policy, Modell, Report, API-Output oder Empfehlung.

Definition 30.1 (Emission). *Eine Emission ist ein Tupel*

$$E = (\text{artifact}, \text{channel}, \text{target}, \text{CrystalRefs}, \text{purpose}, \text{risks}, \text{exit_conditions}). \quad (50)$$

30.2 Exit-Audit

Listing 11: Exit-Audit

```
check purpose_declared
check artifact_has_crystal_refs
check evidence_chain_complete
check replay_bundle_available
check risk_constraints_satisfied
check no_unresolved_high_severity_gaps
check emission_policy_allows_release
if all pass: release
else: hold or fail-closed
```

31 Anwendungsprofile

31.1 Profil A: Software-Synthese

1. Repository und Zielprompt werden zu ProblemSpec.
2. AST, Tests, Cargo/Package-Struktur und README werden als Resonite extrahiert.
3. Architekturentscheidungen werden Dimensionen.
4. Build- und Testpfade werden operative Optionen.
5. Module, Crates und Testharnesses werden Carrier.
6. Solver erzeugt Patches nur unter Projektionskontext.
7. Gate prüft Tests, Lints, deterministische Reproduktion und Crystal-Evidence.

31.2 Profil B: Wissenserschliessung

1. Quellen werden in Claims, Evidenzen, Gegenargumente und Lücken zerlegt.
2. Hypothesen werden Dimensionen.
3. Quellenpfade werden operative Pfade.
4. Betti-Lücken zeigen fehlende Brücken oder zirkuläre Argumente.
5. CrystalSet speichert mehrere plausible Modellfamilien statt falscher Eindeutigkeit.

31.3 Profil C: Resilienz- und Optionalitätsaudit

1. Formale Optionen eines Systems werden inventarisiert.
2. Operative Pfade zu jeder Option werden geprüft.
3. Pfadexzisionen werden als strukturelle Risiken markiert.
4. Carrierdruck und Drift werden gemessen.
5. Anti-Drift-Massnahmen werden simuliert und gegated.

31.4 Profil D: Prozessplanung

1. Ziele, Ressourcen, Abhängigkeiten und Risiken werden als Feld-Cube modelliert.
2. Fiedler-Schnitte erzeugen Arbeitspakete.
3. Carrier-Scheduler verteilt Last auf Teams, Tools oder Phasen.
4. Gate verhindert unbewiesene Milestone-Commits.

32 Roadmap

32.1 Phase 1: Spezifikationshärtung

- Begriffsglossar einfrieren.
- ProblemSpec JSON-Schema definieren.
- Feld-Cube-Kernmodell in Rust implementieren.
- Pfadexzisions- und Optionalitätsmetriken testen.

32.2 Phase 2: PSE-Anbindung

- Observations aus TTF in PSE einspeisen.
- Crystal-IDs zurück in TTF Registry mappen.
- PSE-Falsifier als Gate-Plugin integrieren.
- Pattern Memory für Norm-Promotion verwenden.

32.3 Phase 3: Agent Harness

- LLM-Solver-Trait mit Projektionen versehen.
- Solver-Ausgaben canonicalisieren.
- Replay und Evidence Bundle erzwingen.
- Coagula-Feedback als maschinenlesbare Failure-Delta an Agenten geben.

32.4 Phase 4: Produktisierung

- CLI: `ttf analyze`, `ttf plan`, `ttf solve`, `ttf replay`.
- Web-Gateway für Runs und Crystal Registry.
- Report-Generator für Gap-, Optionalitäts- und Carrier-Audits.
- Benchmark-Suite für Domänenadapter.

33 Masterprompt für Agentenimplementierung

Listing 12: Implementierungs-Masterprompt

```
Du bist ein deterministischer Rust-Systemarchitekt. Implementiere TTF v0.3 als
Workspace mit den Crates ttf-types, ttf-dof, ttf-optionality, ttf-carrier, ttf-
lattice, ttf-dual, ttf-gate, ttf-crystal, ttf-replay und ttf-pse-bridge. Keine
freie Generierung ohne Typen. Jede oeffentliche Funktion braucht Tests. Jeder
Operator braucht eine Registry-Definition mit DeterminismClass, InputType,
OutputType, EvidenceSchema und FailureModes. Implementiere zuerst ProblemSpec,
FieldCube, Dimension, OptionValue, Path, Carrier, Constraint, Resonite, Infogene,
GateResult, CollapseCertificate und CrystalRef. Danach implementiere
DetectPathExcision, WEAVE ordering, CarrierPressure, MigrationScore, MCI und
FailClosedReport. Ziel: cargo test --workspace muss vollstaendig gruen sein.
Keine Netzkabhaengigkeiten im Kern. Keine nichtdeterministischen HashMaps in
canonical outputs. Sortiere alle Sets vor Serialisierung. Jeder Run muss
replaybar sein.
```

A Glossar

Begriff	Bedeutung
Blanker Hypercube	Vorgeformter, aber noch unbelegter Möglichkeitsraum.
Feld-Cube	Erweiterter Hypercube mit Pfaden, Carriern, Krümmung und Evidenz.
Optionalität	Operativ begehbbare Möglichkeiten eines Zustands.
Pfadexzision	Formale Option ohne admittierbaren Pfad.
Carrier	Träger operativer Last, Evidenz, Kontext oder Funktion.
Phase-Ladder	Geordnete Menge phasenversetzter Carrier.
Carrier-Migration	Regelgebundene Lastverlagerung zwischen Carriern.
Anti-Drift	Operator zur Erhaltung von Kohärenz, Attraktorbindung oder Optionalität.
Constraint-Lattice	Verband admissibler Regionen unter Constraints.
Resonite	Messbarer Constraint-Kern.

Infogen	Ausführbares Constraint-Fragment.
WEAVE	Deterministische Priorisierung und Scherung des Constraint-Key-Space.
Dual-Fabric	Primär- und Spiegelsicht auf einen Kandidaten.
MCI	Mirror-Consistency-Index.
Collapse Certificate	Beweisobjekt für deterministischen oder quasi-deterministischen Kollaps.
CrystalSet	Content-addressed Container mehrerer gültiger Lösungen.
Fail-Closed	Sicheres Abbrechen oder Verfeinern statt unbewiesener Commit.

B Quellenbasis dieser Synthese

Diese Fassung absorbiert die vorherige v0.2 des Topologischen Traversierungsframeworks und integriert neutralisierte Mechanismen aus folgenden neu einbezogenen Ressourcen:

- *Gekrümmte Welt: Die Architektur der Unausweichlichkeit.* Verwendet für Optionalität, Pfadexzision, Kompression, Asymmetrie, Verstärkung, Konvergenz und Feldkrümmung.
- *Die Phase-Ladder des Dollar-Systems.* Verwendet für Phase-Ladder, Carrier-Migration, Anti-Drift, Sättigung, Schock-/Frikktionstrigger und Tri-Temporalität.
- *The Physics of Mass Control.* Verwendet für State Algebra, Operator Registry, Constraint-Lattice, Resonite/Infogene, WEAVE, Dual-Fabric, Gating, Fail-Closed, Emissionsaudit und adversarielle Robustheit.
- *Topologisches Traversierungsframework v0.2.* Verwendet als vollständiger Grundstein für Hypercube-Puzzle, 51-Prozent-Perkolation, Wissenserschliessung, negative Topologie, Multi-Solution-Crystals und PSE-Anbindung.

C Maschinenlesbare Kurzspezifikation

Listing 13: TTF v0.3 Minimal Contract

```

TTF_v0_3 := {
  Input: ProblemSpec | ObservationStream | Corpus | ArtifactSet,
  State: FieldCube(D,A,K,G,P,Lambda,kappa,H),
  RequiredOperators: [Project, Solve, Gate, Coagula, Verify, Commit],
  ExtendedOperators: [DetectPathExcision, MigrateCarrier, AntiDrift, WEAVE,
    DualFabricGate],
  Output: Crystal | CrystalSet | GapCrystal | FailClosedReport,
  Guarantees: [ContentAddressed, Replayable, EvidenceBound, GateChecked],
  NonGoals: [CovertManipulation, UnboundedContextClaim, UngatedOracleEmission]
}

```